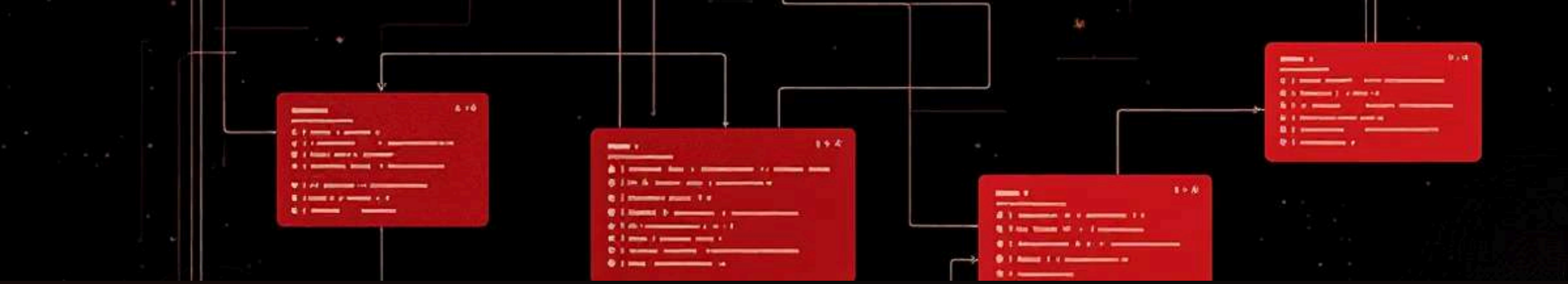
Jenson USA - SQL Data Analysis Project

A comprehensive SQL analysis exploring sales patterns, customer behavior, and product performance across Jenson USA's retail locations.



Project Overview

Learning Objectives

- Master essential SQL query techniques
- Analyze retail business data patterns
- Extract actionable insights from databases
- Apply JOINs, aggregations, and subqueries

Database Scope

- Customer order history and patterns
- Product inventory and sales performance
- Staff members and sales activity
- Store locations and operations

Question 1: Which Store Sold the Most Products?

Understanding store performance helps identify successful locations and replicate best practices across the retail network.

SQL Query: Total Orders by Store

```
SELECT
    stores.store_name,
    SUM(order_items.quantity) AS total_quantity
FROM stores
JOIN orders USING (store_id)
JOIN order_items USING (order_id)
GROUP BY stores.store_name
ORDER BY total_quantity DESC;
```

This query joins three tables to calculate the total quantity of products sold by each store location.

Results: Store Performance Analysis

4,779

Baldwin Bikes

Leading store location with highest unit sales

1,516

Santa Cruz Bikes

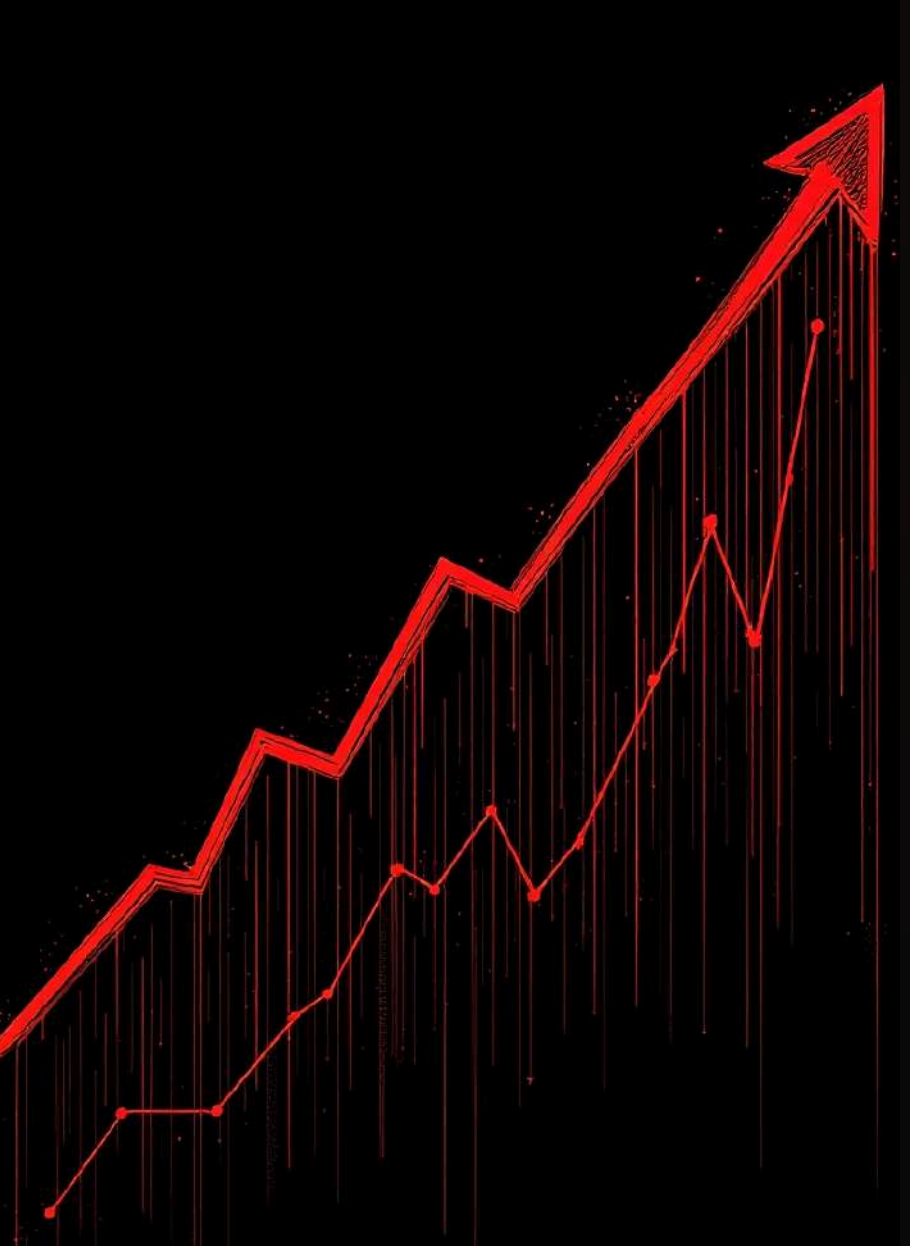
Second-highest performing location

783

Rowlett Bikes

Third in total product sales volume

Key Insight: Baldwin Bikes demonstrates significantly higher customer traffic and overall sales performance, selling more than 3 times the units of Santa Cruz Bikes and over 6 times the units of Rowlett Bikes. This location represents a model for operational excellence.



Question 2: Cumulative Sales Over Time

Window functions allow us to track running totals, revealing product demand patterns and seasonal trends in the bicycle retail market.

SQL Query: Cumulative Sales by Product

This query uses window functions to calculate running totals of product sales over time, partitioned by product name and ordered by date.

```
SELECT
    products.product_name,
    orders.order_date,
    order_items.quantity,
    SUM(order_items.quantity) OVER (
        PARTITION BY product_name
        ORDER BY orders.order_date
    ) AS cumulative_quantity
FROM orders
JOIN order_items USING (order_id)
JOIN products USING (product_id);
```

Query Output: Raw Data

product_name	order_date	quantity	cumulative_quantity
Electra Amsterdam Fashion 3 Ladies - 2017/2018	2018-01-01	1	1
Electra Amsterdam Fashion 3 Ladies - 2017/2018	2018-01-21	2	3
Electra Amsterdam Fashion 3 Ladies - 2017/2018	2018-04-30	2	5
Electra Amsterdam Fashion 3 Ladies - 2017	2017-01-29	2	2
Electra Amsterdam Fashion 3 Ladies - 2017	2017-02-28	1	3
Electra Amsterdam Fashion 3 Ladies - 2017	2017-03-03	1	4
Electra Amsterdam Fashion 3 Ladies - 2017	2017-03-09	2	6
Electra Amsterdam Fashion 3 Ladies - 2017	2017-04-06	1	7
Electra Amsterdam Fashion 3 Ladies - 2017	2017-04-15	2	9
Electra Amsterdam Fashion 3 Ladies - 2017	2017-04-16	1	10
Electra Amsterdam Fashion 3 Ladies - 2017	2017-06-27	2	14

This table provides the detailed raw data for the cumulative sales of the "Electra Amsterdam Fashion 3 Ladies" product, as calculated by the SQL query using window functions.

Question 3: Highest Total Sales Product per Category

```sql
WITH ProductTotalSales AS (
    SELECT
        p.product_name,
        p.category_id,
        SUM(od.quantity * p.list_price) AS total_sales
    FROM
        Products p
    JOIN
        OrderDetails od ON p.product_id = od.product_id
    GROUP BY
        p.product_name, p.category_id
),
RankedProductSales AS (
    SELECT
        product_name,
        category_id,
        total_sales,
        ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY total_sales DESC) as rn
    FROM
        ProductTotalSales
)
SELECT
    product_name,
    category_id,
    total_sales
FROM
    RankedProductSales
WHERE
    rn = 1;
```

This SQL query identifies the top revenue-generating product within each category. It first calculates the total sales for each product by multiplying the quantity sold by its list price. Then, it uses a Common Table Expression (CTE) and the `ROW_NUMBER()` window function, partitioned by `category_id` and ordered by `total_sales` in descending order, to rank products within their respective categories. Finally, it selects only the product with rank 1 (the highest total sales) from each category.

Question 3: Output - Top Revenue Products by Category

category_id	product_name	total_sales	highest_sale
1	Electra Girl's Hawaii 1 (20-inch) - 2015/2016	46198486.00	46198486.00
2	Electra Townie Original 7D EQ - 2016	8039866.00	8039866.00
3	Electra Townie Original 7D EQ - 2016	9359844.00	9359844.00
4	Surly Straggler 650b - 2016	25382549.00	25382549.00
5	Trek Conduit+ - 2016	43499855.00	43499855.00
6	Trek Slash 8 275 - 2016	61598846.00	61598846.00
7	Trek Domane SLR 9 Disc - 2017	23649957.00	23649957.00

These products represent the highest revenue generators in their respective categories and should be prioritized for inventory and marketing.

Question 4: Customer with Highest Spending

This query uses JOINs to connect customer, order, and order_item tables, calculating total sales by multiplying quantity and list price. It then identifies the most valuable customer by total spending.

```
SELECT
    customers.first_name,
    customers.last_name,
    SUM(order_items.quantity * order_items.list_price) AS total_sales
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id
JOIN order_items ON orders.order_id = order_items.order_id
GROUP BY customers.customer_id, customers.first_name,
customers.last_name
ORDER BY total_sales DESC
LIMIT 1;
```

Question 4: Output - Top Spending Customer

customer_id	full_name	total_sales
10	Pamelia Newman	3780184.00

Pamelia Newman is the highest-spending customer with total purchases of $3,780,184.00, making her a VIP customer for the business.

Question 5: Highest-Priced Product per Category

This query uses a CTE (WITH clause) to partition products by category_name and find the MAX list_price for each category, then selects all columns where list_price equals the highest_price. This identifies the premium product in each category.

```
WITH CategoryHighestPrice AS (
    SELECT
        product_id,
        product_name,
        list_price,
        category_id,
        category_name,
        MAX(list_price) OVER (PARTITION BY category_name) AS
highest_price_in_category
    FROM products
)
SELECT
    product_name,
    category_name,
    list_price
FROM CategoryHighestPrice
WHERE list_price = highest_price_in_category
ORDER BY category_name, list_price DESC;
```

Question 5: Output - Premium Products by Category

category_name	product_name	list_price	highest_price
Children Bicycles	Trek Superfly 24	48999.00	48999.00
Children Bicycles	Electra Straight 8 (20-inch) - Boy's	48999.00	48999.00
Comfort Bicycles	Electra Townie 3 EQ (20-inch) - Boy's	48999.00	48999.00
Cruisers Bicycles	Electra Townie Commute Go!	299999.00	299999.00
Electric Bikes	Trek Powerfly 8 FS Plus	499999.00	499999.00
Mountain Bikes	Trek Fuel EX 9.8 275 Plus	529999.00	529999.00
Road Bikes	Trek Domane SLR 9 Disc	1199999.00	1199999.00

This table displays the premium offerings in each product category. These products represent the highest-priced items, indicating top-tier models and specialized editions within their respective segments.

Question 6: Total Orders by Customer per Store

This query uses JOINs to connect customers, orders, and stores tables, then uses CONCAT to combine first_name and last_name, and COUNT to count total orders. It groups by customer_id and store_name. This shows customer shopping patterns across different store locations.

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    s.store_name,
    COUNT(o.order_id) AS total_orders
FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
JOIN stores AS s ON o.store_id = s.store_id
GROUP BY c.customer_id, full_name, s.store_name
ORDER BY c.customer_id, total_orders DESC;
```

Question 6: Output - Customer Orders by Store

customer_id	full_name	store_name	total_orders
259	Johnathan Velazquez	Santa Cruz Bikes	1
175	Nova Hess	Santa Cruz Bikes	2
60	Neil Merrill	Santa Cruz Bikes	2
91	Marvin Mullins	Santa Cruz Bikes	2
258	Maribel William	Santa Cruz Bikes	1
552	Lea Key	Santa Cruz Bikes	1
1175	Sindy Anderson	Santa Cruz Bikes	1
541	Lanita Burton	Santa Cruz Bikes	1
696	Norine Huffman	Santa Cruz Bikes	1
923	Randee Pitts	Santa Cruz Bikes	1
1035	Tangela Hurley	Santa Cruz Bikes	1
1149	Drizella Gilliam	Santa Cruz Bikes	1

This data reveals customer loyalty and shopping frequency at each store location.

Question 7: Staff Members with No Sales

This query uses a LEFT JOIN between staff and orders tables, then filters WHERE orders.order_id IS NULL to find staff members who haven't made any sales. This identifies underperforming or newly hired staff who may need additional training or support.

```
SELECT
    s.staff_id,
    s.first_name,
    s.last_name
FROM staff AS s
LEFT JOIN orders AS o ON s.staff_id = o.staff_id
WHERE o.order_id IS NULL;
```

Question 7: Output - Staff Members with No Sales

staff_id	staff_name
1	Fabiola Jackson
4	Virgie Wiggins
5	Jannette David
10	Bernardine Houston

This table lists staff members who have not yet recorded any sales. These individuals may require additional training, support, or a performance review to understand the reasons behind their lack of sales.

Question 8: Top 3 Most Sold Products by Quantity

This query uses JOINs to connect products and order_items tables, then uses SUM to calculate total quantity sold, groups by product_name, orders by total quantity DESC, and limits to 3 results. This identifies the best-selling products that drive revenue and customer satisfaction.

```
SELECT
    p.product_name,
    SUM(oi.quantity) AS total_quantity_sold
FROM products AS p
JOIN order_items AS oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY total_quantity_sold DESC
LIMIT 3;
```

Question 8: Output - Top 3 Most Sold Products

product_id	product_name	total_sold
6	Surly Ice Cream Truck Frameset - 2016	167
13	Electra Cruiser 1 (24-Inch) - 2016	157
16	Electra Townie Original 7D EQ - 2016	156

This table highlights the top-performing products by quantity sold, indicating strong customer demand and market preference for these models. These items could be prioritized for inventory management, marketing campaigns, and future product development.

Question 9: Find the Median Value of Price List

This query calculates the median list price from the products table. It utilizes window functions to assign row numbers in both ascending (`rn_asc`) and descending (`rn_desc`) order based on `list_price`. It also calculates the total count of products. By identifying the row(s) where the ascending and descending ranks are equal or consecutive, it accurately pinpoints the middle value(s) of the dataset. Finally, it computes the average of these middle value(s) to derive the true median, handling both odd and even numbers of products in the list.

```
WITH ProductPrices AS (
    SELECT
        list_price,
        ROW_NUMBER() OVER (ORDER BY list_price ASC) AS rn_asc,
        ROW_NUMBER() OVER (ORDER BY list_price DESC) AS rn_desc,
        COUNT(*) OVER () AS total_count
    FROM products
),
MedianCandidates AS (
    SELECT
        list_price
    FROM ProductPrices
    WHERE rn_asc = rn_desc OR rn_asc = rn_desc + 1 OR rn_desc = rn_asc + 1
)
SELECT
    AVG(list_price) AS median_list_price
FROM MedianCandidates;
```

Question 9: Output - Median Price List Value

median
74999.00

This value represents the median list price across all products, indicating the central price point after accounting for all product offerings in the dataset.

Question 10: Products Never Ordered

This query uses NOT EXISTS to find products that have never been ordered. It selects product_id and product_name from products where there are no matching order_items records. This identifies inventory items that may need marketing attention or discontinuation.

```
SELECT
    p.product_id,
    p.product_name
FROM products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items AS oi
    WHERE oi.product_id = p.product_id
);
```

Question 10: Output - Never Ordered Products

product_id	product_name
1	Trek 820 - 2016
121	Surly Krampus Frameset - 2018
125	Trek Kids' Dual Sport - 2018
154	Trek Domane SLR 6 Disc Women's - 2018
195	Electra Townie Go! 8i Ladies' - 2018
267	Trek Precaliber 12 Girl's - 2018
284	Electra Savannah 1 (20-inch) - Girl's - 2018
291	Electra Sweet Ride 1 (20-inch) - Girl's - 2018
316	Trek Checkpoint ALR 4 Women's - 2019
317	Trek Checkpoint ALR 5 - 2019
318	Trek Checkpoint ALR 5 Women's - 2019
319	Trek Checkpoint SL 5 Women's - 2019
320	Trek Checkpoint SL 6 - 2019
321	Trek Checkpoint ALR Frameset - 2019

These products currently have zero sales, indicating they might be new stock, unpopular items, or products that have not been adequately promoted. This insight suggests a need for promotional strategies, inventory review, or potentially discontinuation if they continue to underperform.

Question 11: Top Performing Staff Members

This query uses a CTE to calculate total_sales for each staff member using LEFT JOIN and COUNT, then selects staff members whose total_sales is greater than the average total_sales across all staff. This identifies high-performing sales staff who exceed average performance.

```
WITH StaffSales AS (
    SELECT
        s.staff_id,
        s.first_name,
        s.last_name,
        COUNT(o.order_id) AS total_sales
    FROM staff AS s
    LEFT JOIN orders AS o ON s.staff_id = o.staff_id
    GROUP BY s.staff_id, s.first_name, s.last_name
)
SELECT
    ss.staff_id,
    ss.first_name,
    ss.last_name,
    ss.total_sales
FROM StaffSales AS ss
WHERE ss.total_sales > (SELECT AVG(total_sales) FROM StaffSales);
```

Question 11: Output - Above Average Performing Staff

staff_id	staff_name	total_sales
2	Mireya Copeland	164
3	Genna Serrano	184
6	Marcelene Boyer	553
7	Venita Daniel	540

These staff members have exceeded the average sales performance and should be recognized as top performers.

Question 12: Customers Who Ordered All Product Categories

The query uses a CTE to join customers with orders, order_items, and products, then counts DISTINCT category_id per customer. It uses HAVING to filter customers whose category_orders count equals the total number of categories. This identifies customers with the most diverse purchasing behavior across all product categories.

```
WITH CustomerCategoryCounts AS (
    SELECT
        c.customer_id,
        c.first_name,
        c.last_name,
        COUNT(DISTINCT p.category_id) AS distinct_categories_ordered
    FROM customers AS c
    JOIN orders AS o ON c.customer_id = o.customer_id
    JOIN order_items AS oi ON o.order_id = oi.order_id
    JOIN products AS p ON oi.product_id = p.product_id
    GROUP BY c.customer_id, c.first_name, c.last_name
)
SELECT
    ccc.customer_id,
    ccc.first_name,
    ccc.last_name,
    ccc.distinct_categories_ordered
FROM CustomerCategoryCounts AS ccc
WHERE ccc.distinct_categories_ordered = (SELECT COUNT(DISTINCT category_id) FROM products);
```

Question 12: Output - Omnicategory Customers

customer_id	category_orders
9	7

Customer ID 9 is the only customer who has ordered from all 7 product categories, making them a highly valuable omnichannel customer with diverse purchasing interests.

SQL Concepts Mastered

JOINs

Combine related tables using INNER, LEFT, and other join types to connect customers, orders, products, and staff data.

GROUP BY & Aggregations

Summarize data with SUM, COUNT, and AVG functions to calculate totals and identify patterns.

HAVING & WHERE

Filter results before and after aggregation to isolate specific data subsets and answer complex questions.

Subqueries & EXISTS

Use nested queries to perform comparative analysis and detect relationships between data sets.

Thank you for exploring SQL data analysis with Jenson USA! These techniques form the foundation for extracting meaningful business insights from any database.

Thank You

For exploring SQL Data Analysis with Jenson USA

Presented by: Chandra Prakash Choudhary

LinkedIn Profile

Connect with me on LinkedIn for more insights on data analysis and SQL optimization